gradiant

AVP2: Requisitos funcionales y técnicos

Alejandra Ares Rivas, Paula Esmorís Rodriguez, María Gómez Fernández

USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

Proyecto	Mejorar la detección de aeronaves de ala fija en un sistema DAA.
Cliente	Gradiant (Centro Tecnolóxico de Telecomunicacións de Galicia)
Período	01/03/2026 - 12/05/2026
Equipo	Alejandra Ares Rivas, María Gómez Fernández, Paula Esmorís Rodríguez
Versión	1.0v

Índice

Definición del alcance del proyecto	4
Requisitos funcionales	5
Requisitos no funcionales	9
Arquitectura e ingeniería	10
Conjunto de datos (Dataset)	13

Definición del alcance del proyecto

En cuanto al alcance del proyecto, a continuación se definirá lo que se considerará un éxito en el proyecto, así como las tareas que no se incluyen en el mismo y que se quedan fuera de la responsabilidad de las integrantes.

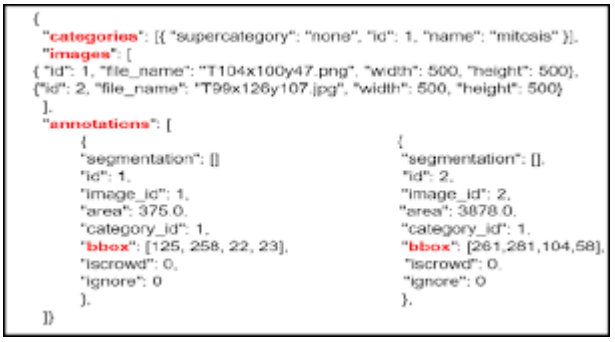
El principal objetivo del proyecto es la optimización de los sistemas Detect and Avoid (DAA) de la empresa mediante la integración de **datos sintéticos** en el modelo de IA utilizado. Para cumplir con este objetivo, hay varias tareas que se deben completar:

1. **Creación de tres escenarios manualmente en los que aparezca una aeronave** mediante la herramienta de modelado 3D **Blender**.





2. **Generación a escala en el CESGA** a partir de cada uno de los escenarios de Blender, en los que se modifican parámetros (luz, distancia, ángulo, condiciones meteorológicas) mediante la API de Blender en Python.
    a. En el mismo script de generación de imágenes variadas se realizará un **etiquetado automático en formato COCO** (Common Objects in Context), que es el que necesita el modelo de IA usado para procesar las imágenes.



3. Una vez obtenido el dataset completo y variado, se hará una **validación del modelo YOLO-X** proporcionado por la empresa.
4. Al finalizar la evaluación del modelo con las imágenes sintéticas, se hará una **comparación de las métricas** que ha obtenido la empresa tras entrenar el modelo con imágenes reales y los resultados obtenidos tras el entrenamiento con el dataset sintético.

En el alcance del proyecto quedan **excluidas** tareas como:

- ➔ La **captura de imágenes reales**.
- ➔ El **reentrenamiento del modelo** total desde cero.
- ➔ El **despliegue de hardware** en producción.
- ➔ **Soporte técnico** después de la entrega final.

Centrándonos ahora en las suposiciones sobre el proyecto, se asumirá:

- ➔ La corrección del script de Python en Blender a la hora de **traducir correctamente los datos** del archivo *.blend* a anotaciones COCO sin errores en las bounding boxes.
- ➔ Que la **variación de parámetros** en los tres escenarios creados es suficiente para que el modelo YOLO-X mejore su robustez.
- ➔ La **disponibilidad del entorno de computación del CESGA** para poder renderizar las imágenes en un tiempo razonable.
- ➔ El suministro por parte de la empresa de su **modelo YOLO-X** empleado en sus proyectos y la facilitación de las instrucciones correspondientes para su respectivo uso.
- ➔ El suministro de la empresa al equipo del proyecto de las **métricas de rendimiento de su modelo con imágenes reales** para poder hacer así la comparativa posterior correspondiente con los resultados obtenidos durante el proyecto y poder tener una validez estadística.

En la realización del proyecto también se deben tener en cuenta las dependencias técnicas existentes. En el caso de este proyecto, las dependencias son:

- ➔ **Infraestructura del CESGA**: para generar un conjunto de tamaño aceptable de imágenes en un tiempo razonable, se necesita la disponibilidad de nodos de computación y de colas de trabajo en el CESGA.
- ➔ **Modelo de la empresa**: para poder hacer la validación del dataset generado, se necesita la entrega del modelo YOLO-X y la documentación necesaria para poder manipularlo por parte de la empresa tutora.
- ➔ **Compatibilidad de los datos**: para poder usar con éxito el modelo YOLO-X es necesario que las etiquetas de los datos generadas en formato COCO sean correctamente procesables por el modelo de evaluación directamente.
- ➔ **Métricas obtenidas por la empresa**: para poder finalizar con éxito el proyecto, se necesitan las métricas obtenidas por la empresa para poder hacer una comparativa correcta con las métricas obtenidas en este proyecto y poder ver si existe mejora o no.

Requisitos funcionales

Los requisitos funcionales son las acciones específicas que el sistema debe hacer una vez finalizado el proyecto. En este proyecto se destacan los siguientes requisitos funcionales:

En cuanto a la generación de los datos

- ➔ El equipo del proyecto debe **generar escenarios en Blender lo más realistas posible**, en entornos naturales y en los que aparezca una aeronave, la cual se pueda colocar fácilmente en muchos puntos del escenario.

- ➔ El sistema debe generar un **conjunto grande y variado de imágenes sintéticas** mediante el software de modelado 3D Blender.
- ➔ El **script de Python** debe ser capaz de modificar aleatoriamente y de forma controlada los parámetros de iluminación, posición de aeronave, condiciones meteorológicas y distancias respecto a la cámara mediante la API de Blender en Python.

En cuanto al etiquetado automático de las imágenes

- ➔ El sistema debe **generar automáticamente** los archivos JSON con las anotaciones sobre las imágenes en **formato COCO** al mismo tiempo en el que se generan las imágenes del dataset.
- ➔ Las etiquetas deben incluir obligatoriamente las **bounding boxes** (rectángulo que delimita el área que ocupa la aeronave), la posición relativa al horizonte y la distancia exacta entre la cámara y el objetivo.
- ➔ Se debe **identificar de forma automática el modelo de aeronave** de la imagen, así como las condiciones meteorológicas de la imagen en cuestión.

En cuanto a la evaluación y la calidad de los resultados

- ➔ El dataset sintético generado debe estar **correctamente etiquetado y generado** para poder aplicarlo al modelo YOLO-X de la empresa sin complicaciones.
- ➔ Se deben extraer métricas comparativas de rendimiento como la **Precisión, el Recall o el F1-score** para poder comparar los resultados con las métricas obtenidas por la empresa y medir la mejora.
- ➔ Las métricas de rendimiento del modelo YOLO-X **deben ser mejores con la incorporación de los datos sintéticos** que con solo las imágenes reales.

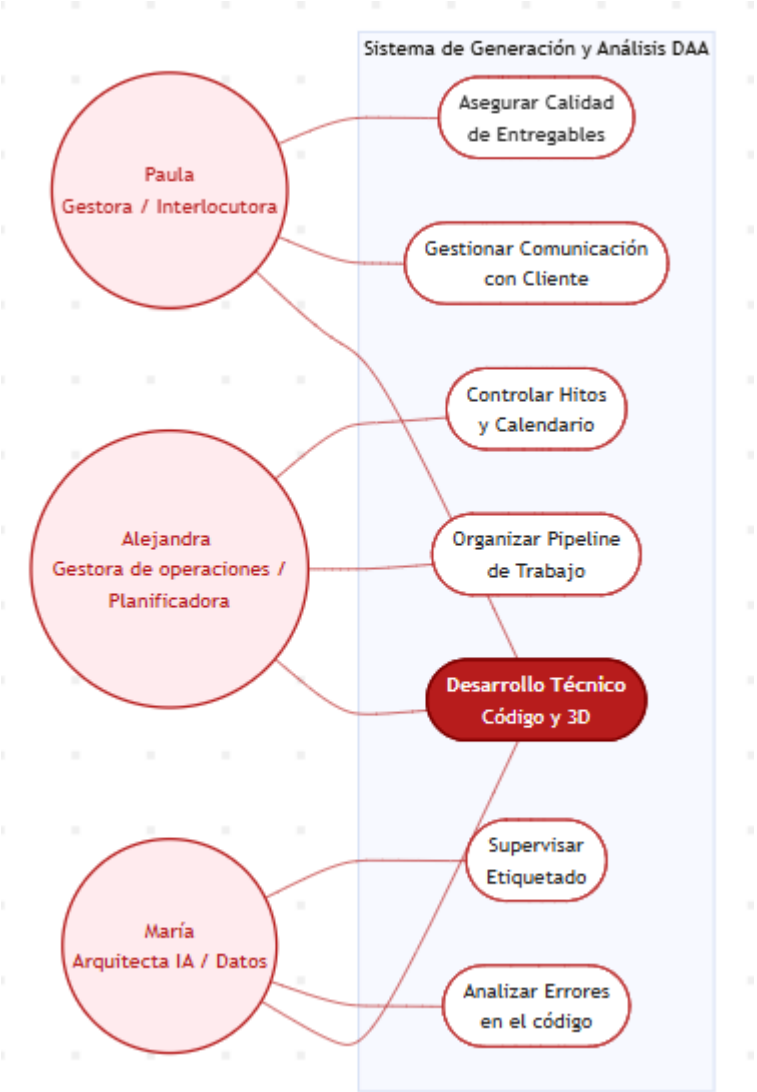
Historias de usuario (user stories)

Rol	Historia de usuario	Criterio de aceptación
Paula (Gestora)	Como gestora del proyecto, quiero asegurar que la generación de los datos cumple con las normativas correspondientes y los límites de alcance definidos.	El documento debe tener una parte centrada en la ética y privacidad además de una definición clara de los límites del proyecto aprobada por la empresa.
Alejandra (Operadora)	Como responsable de operaciones, quiero planificar con detalle la ejecución masiva en el CESGA para garantizar que la generación de las imágenes sea eficiente y no sufra bloqueos por falta de recursos	Supervisión de la ejecución de los scripts de generación del CESGA y revisión de dichos scripts.
María	Como Arquitecta de IA, quiero analizar	Informe de métricas

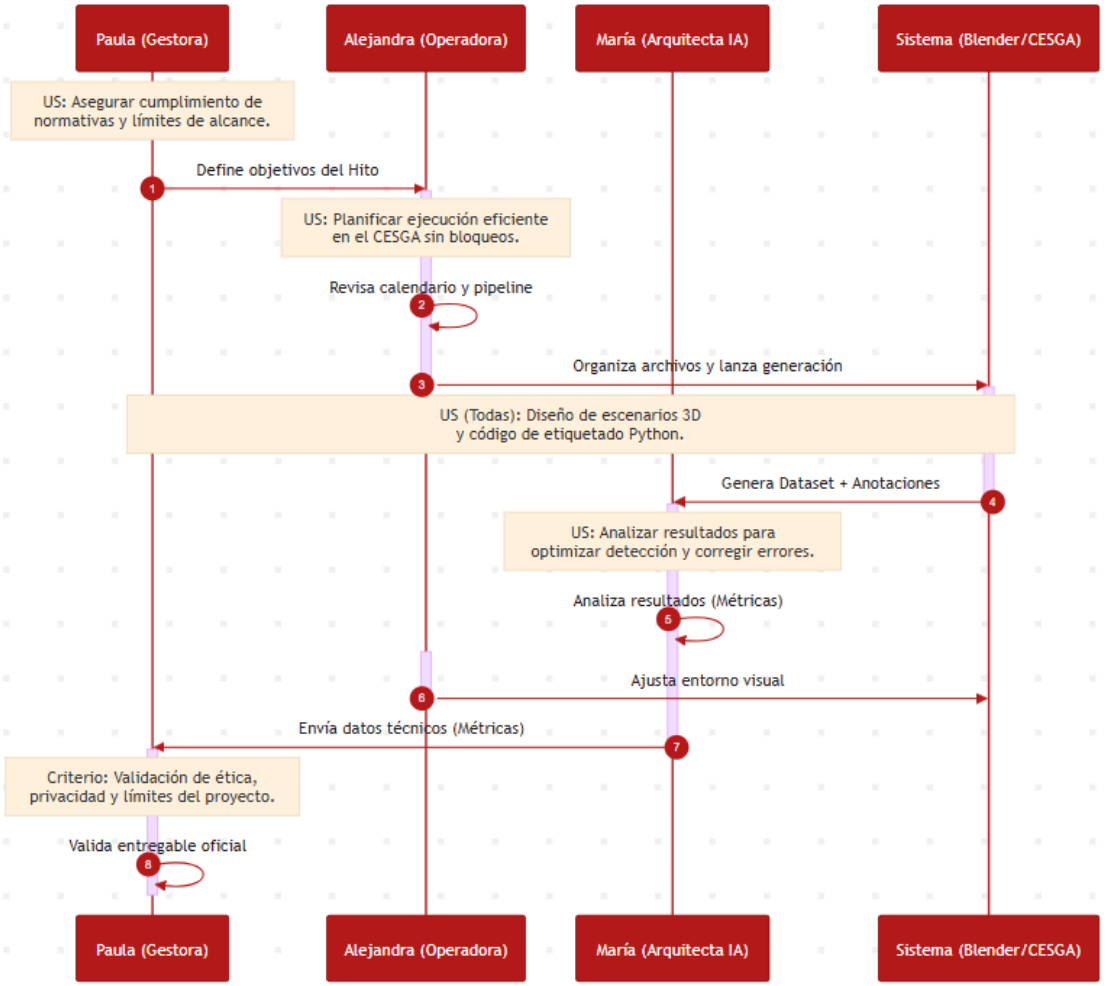
(Arquitecta de IA)	los resultados del entrenamiento y ajustar parámetros (luz/distancia), para optimizar la detección y corregir posibles errores.	comparativas y propuesta de ajustes de parámetros para el siguiente ciclo.
Todas (Desarrolladoras)	Como Desarrolladoras, queremos escribir el código Python de etiquetado y diseñar los escenarios 3D, para generar el dataset necesario para el entrenamiento	Script funcional en el CESGA y 3 escenarios completados.

Diagramas UML

- **Diagrama de caso de uso:** el diagrama ilustra el modelo de trabajo que se está siguiendo, en el que se puede ver que las tareas de desarrollo son una responsabilidad compartida, mientras que para el resto de tareas tenemos roles especializados para cada integrante del grupo.



- **Diagrama de secuencia (Flujo de trabajo):** este diagrama detalla el flujo temporal y técnico desde la configuración del entorno hasta la obtención de métricas de validación, mostrando las tareas que tiene cada miembro del equipo de forma individual y sus respectivas historias de usuario, teniendo en cuenta el hecho de que todas participarán en el desarrollo del proyecto.



Requisitos no funcionales

Los requisitos no funcionales son las restricciones o estándares de calidad que marcan el comportamiento del sistema. En este caso como nuestro objetivo es el crear un buen dataset, estos requisitos tratan de evaluar la calidad, utilidad y rigor técnico de los datos que vamos a generar, garantizando que el dataset es un instrumento de prueba fiable y útil para el modelo. En este proyecto se destacan los siguientes requisitos no funcionales:

En cuanto a la calidad y precisión del etiquetado

- ➔ Las imágenes deben contener los **datos del entorno** (clima, distancia, ambiente) **correctamente mapeados** en el JSON del etiquetado.
- ➔ **Precisión de las *Bounding Boxes*** respecto a la silueta de la aeronave en las etiquetas, con un error de píxel nulo, garantizado por el script de python que extrae las coordenadas de Blender.
- ➔ Las imágenes no deben presentar **fallos visuales** o incongruencias fruto del renderizado como posiciones imposibles de la aeronave o texturas mal cargadas.

En cuanto a la representatividad de los datos y mitigación sesgos

- ➔ **Número equilibrado de escenarios** de cada tipo para tener una evaluación justa y poder obtener medias de precisión equitativas.
- ➔ Debe incluir **aeronaves diferentes** y en **diferentes ángulos de vuelo** (visto desde diferentes puntos), además de que la cámara debe situarse también en diferentes partes del escenario.
- ➔ Añadir **diversidad** de dificultad al dataset con casos críticos (gran distancia o poca visibilidad) para evaluar los límites operativos del modelo.

En cuanto a la eficiencia, usabilidad y rendimiento.

- ➔ Asegurar la **estandarización** al garantizar la compatibilidad del dataset con el estándar COCO para poder ser utilizado en otros sistemas o frameworks de IA (escalabilidad).
- ➔ La estructura del JSON (COCO) debe estar **indexada** de tal forma que cualquier herramienta de validación pueda realizar consultas de filtrado.
- ➔ Las imágenes deben tener una **resolución constante de 1920x1080** para que las métricas de rendimiento del modelo sean comparables entre sí y para evitar que la IA tenga que reescalar.

En cuanto a ética, privacidad y seguridad

- ➔ El dataset no debe contener ningún recurso en sus imágenes que tenga derechos de autor o datos de carácter personal como rostros o propiedades privadas, garantizando el **cumplimiento del RGPD.** Debe ser 100% sintético.
- ➔ Cada imagen debe ser **reproducible y verificable** mediante el registro de los parámetros del renderizado (semilla y configuración de la escena), garantizando la trazabilidad total de las condiciones utilizadas para su generación.

Arquitectura e ingeniería

Como ya se mencionó anteriormente en el documento, las principales fases del proyecto son la creación y etiquetado de datos sintéticos mediante Blender, la automatización de este proceso y la validación del modelo de Machine Learning YOLO-X creado y desarrollado por la empresa Gradiant.

Stack tecnológico

- ➔ El principal lenguaje a usar en todo el proyecto es **Python**, junto con el uso de archivos **JSON** para el almacenamiento de anotaciones.
- ➔ En cuanto al renderizado de imagen, principalmente utilizamos la interfaz gráfica de **Blender** para la creación manual de los escenarios base y posteriormente el proceso de renderizado se automatiza mediante la **API de Blender en Python,** permitiendo la generación masiva de datos.
- ➔ Para la incorporación de modelos 3D y recursos adicionales necesarios para la creación de escenarios, se utiliza el complemento **BlenderKit.**
- ➔ El modelo de machine learning es un modelo YOLO-X que se basa en **PyTorch**.
- ➔ Para la automatización utilizamos dos archivos: un script de **Python** que usa la API de Blender para la generación de escenarios y anotaciones automáticas y un script de **SLURM** *.sh* para lanzar la ejecución en el supercomputador Finisterrae del **CESGA** en modo batch y paralelizar usando GPU, agilizando así esta tarea.
- ➔ El formato de datos final son imágenes renderizadas en *png* con los diversos escenarios creados y su correspondiente JSON con las anotaciones en formato **COCO**.

Gestión de datos

El primer paso a realizar es la creación de los escenarios base desde cero utilizando las propias herramientas de modelado y creación de Blender.

Para facilitar la creación de escenarios y tener acceso a recursos ya creados por otros usuarios de manera gratuita hemos usado el complemento para blender **BlenderKit.** Este permite acceder a una biblioteca online de recursos 3D directamente dentro del programa. Algunos ejemplos de uso es la descarga de modelos 3D, como puede ser alguna avioneta, o el uso de HDRIs para iluminación más realista.

Posteriormente se suben los archivos al supercomputador del CESGA y un script en Python ejecuta Blender en modo background, carga la escena base y aplica variaciones aleatorias sobre diferentes parámetros como la posición y rotación del objeto, la iluminación o la orientación de la cámara.

A continuación, se realiza el renderizado automático de las imágenes.

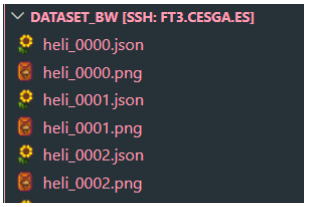
De forma simultánea se generan las anotaciones en formato JSON utilizando el estándar de COCO.

Hay cuatro posibles tipos de objeto a identificar: UFO, avión, pájaro o helicóptero.

A parte de eso también se tienen en cuenta las características de las imágenes (como la altura y el ancho de esta) y otras anotaciones específicas del escenario, como las condiciones meteorológicas (si hay niebla, si está lloviendo...), las coordenadas del bounding box o el modelo del avión, entre otras.

Para la limpieza de los datos se evitarán y eliminarán frames donde la aeronave esté completamente oculta por el terreno o fuera de la vista de la cámara.

La estructura final del dataset es una carpeta **dataset_bw** con todas las imágenes en formato png y sus correspondientes JSON.



Generación automática de etiquetas

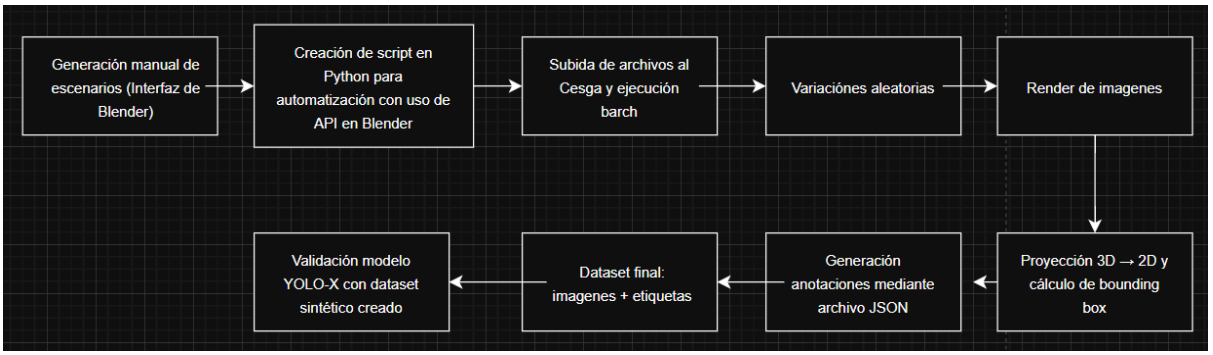
El sistema utiliza la API de Blender *bpy* para calcular automáticamente las bounding boxes a partir de las coordenadas 3D del objeto.

Esto se realiza mediante el uso de la posición de la cámara en el archivo de blender *.blend* para la transformación de coordenadas mundo a una proyección a espacio 2D y así calcular el bounding box con precisión.

```
"bbox": get_bbox(heli, camara),
```

Esto elimina la necesidad de anotación manual, reduciendo costes y aumentando la escalabilidad del sistema.

El pipeline de datos



Integraciones y APIs

La principal herramienta y el núcleo de nuestro sistema es la **API de Blender.** Esta se utiliza principalmente para:

- ➔ Modificar la escena (posición, rotación, iluminación).
- ➔ Controlar la cámara.
- ➔ Renderizar imágenes.
- ➔ Generar anotaciones automáticamente.

Otra parte importante es la **ejecución automatizada en el CESGA**. El script SLURM ejecuta el script de Python en modo batch para generar miles de imágenes y paralelizar la ejecución utilizando el supercomputador del CESGA.

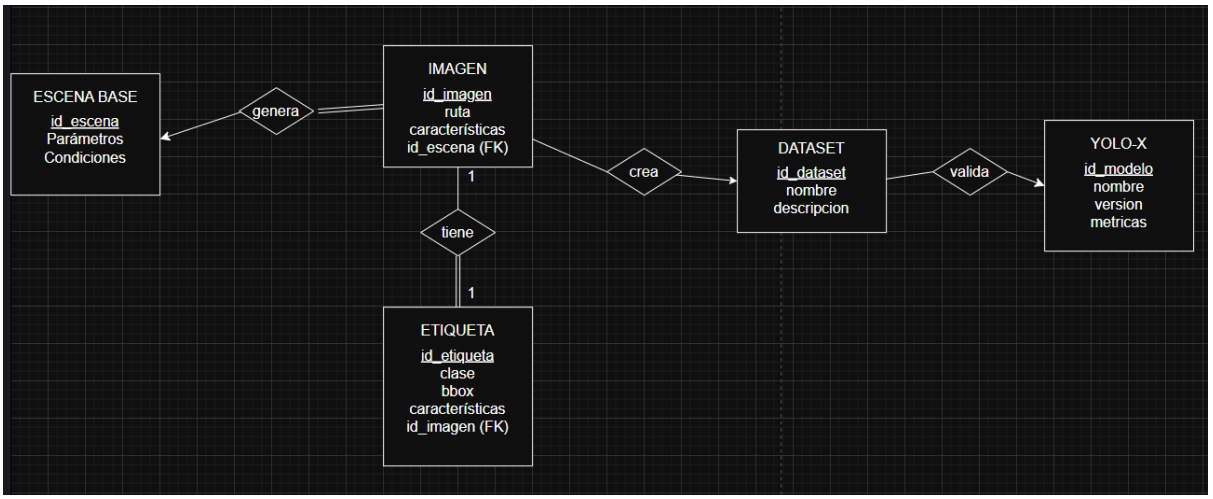
```
# EJECUTAMOS USANDO LA RUTA COMPLETA AL EJECUTABLE DE BLENDER
./blender-5.0.1-linux-x64/blender -b "cottage2.blend" -noaudio -P "generar2.py" -- $INICIO $FIN
```

Por último, se emplea el modelo **YOLOX**, integrando el dataset sintético generado con el objetivo de evaluar su capacidad para detectar aeronaves a partir de datos artificiales. De este modo, se analiza la eficacia del modelo, originalmente desarrollado por la empresa, en escenarios sintéticos, permitiendo validar su rendimiento y generalización

El propio modelo YOLO-X utiliza otra API para la inicialización del conjunto de datos COCO. Los datos de anotación se cargan en memoria mediante la **API de COCO.** Los argumentos son:

- ➔ **data_dir (str):** directorio raíz del conjunto de datos.
- ➔ **json_file (str):** nombre del archivo JSON de COCO.
- ➔ **name (str):** nombre de los datos COCO (por ejemplo, 'train2017' o 'val2017').
- ➔ **img_size (int):** tamaño objetivo de la imagen después del preprocesamiento.
- ➔ **preproc:** estrategia de aumento de datos.

Diagrama Entidad-Relación



Conjunto de datos (Dataset)

En este proyecto el conjunto de datos o dataset es el núcleo del proyecto pues nos centramos en generar un dataset idóneo para poder probar el modelo que nos proporciona la empresa y así observar cómo de bueno es realmente su modelo, pudiendo ofrecerles feedback sobre posibles mejoras o información útil respecto a los tipos de escenarios, condiciones meteorológicas específicas, distancias de la cámara a la aeronave o incluso iluminaciones donde más falla su modelo.

Para la generación de estos datos usamos Blender, ahí generamos escenarios diferentes donde junto con modelos 3D de aeronaves podemos crear imágenes realistas. El dataset consta de 500 imágenes con diferentes escenarios base, ambientes y condiciones e iluminación.

Distribución por escenarios: dividiremos las 500 imágenes en tres bloques principales según el escenario.

- ➔ Mar: 166 imágenes
- ➔ Campo: 167 imágenes
- ➔ Montaña: 167 imágenes

Ambientes y condiciones meteorológicas: Dentro de cada escenario (166-167 imágenes) haremos modificaciones en la meteorología, distancia de la aeronave e incluso iluminación para tratar de generar situaciones más exigentes.

1. **Condiciones meteorológicas**
    a. Despejado: 70 imágenes (luz y visibilidad clara)
    b. Niebla: 64 imágenes donde trataremos de probar diferentes niveles de niebla y visibilidad para probar el límite detección.
    c. LLuvia: 33 imágenes (Usando *particles* en Blender)

2. **Iluminación y movimiento:** Dentro de cada subgrupo de imágenes trataremos de cambiar la iluminación y el contraste. Incluso el enfoque de la aeronave para simular movimiento.
3. **Distancia:** En cada una de las 500 fotos aleatorizaremos la posición de la aeronave para conseguir tanto distancias cortas como muy largas, donde el avión ocupe escasos píxeles. Esto nos permitirá testear la sensibilidad del modelo.

Analíticas:

A partir del dataset y los resultados obtenidos del modelo podremos llevar a cabo un desglose analítico por etiquetas como un análisis de sensibilidad a partir de la precisión media en función del entorno o la identificación de la distancia mínima de detección efectiva a partir de la cual el modelo pierde confianza o baja de accuracy.

Ética y Privacidad:

Dado que nuestro proyecto utiliza datos sintéticos generados por nosotras, la conformidad con el **Reglamento General de Protección de Datos** se simplifica, el dataset deja de estar sujeto a la mayoría de sus restricciones, reduciendo el riesgo legal y los costes de gestión de privacidad. Al utilizar imágenes generadas en Blender se garantiza que no se usan rostros humanos, matrículas de coches o propiedades privadas identificables, eliminando así todo riesgo de procesar datos con carácter personal. Además, siguiendo el RGPD y la Ley de la IA de la UE que exigen modelos explicables y transparentes, con nuestro dataset además de las imágenes tenemos un archivo JSON en formato COCO con el etiquetado que garantiza la trazabilidad del modelo en caso de fallo pudiendo especificar qué datos se usaron y de qué tipo para probarlo.

Confirmación y Firma del cliente:

Firmado por JOVE DIAZ DANIEL - ***8448** el día 08/04/2026 con un certificado emitido por AC FNMT Usuarios